

NETWORK PROGRAMMING & SECURITY

UNIT 1 — COMPREHENSIVE EXAM NOTES

TCP/IP Protocols • Sockets API • Socket Functions • Byte Ordering • Address Conversion

SECTION 1: INTRODUCTION — PROTOCOLS & CLIENT-SERVER

1.1 What is a Protocol?

A protocol is an agreement on how programs will communicate over a network. It defines:

- Which program initiates communication
- When responses are expected
- The format and sequence of messages exchanged

Example: Web server and web client communicate using HTTP. The client sends a request, the server responds with the resource.

1.2 Client–Server Architecture

Most network-aware applications use client–server communication:

- Client: initiates the connection (active open); communicates with only one server at a time
- Server: waits for connections (passive open); communicates with many clients simultaneously

KEY POINT

The client and server are typically user processes. The TCP and IP protocol layers are part of the protocol stack inside the OS kernel.

The client-server application may be thought of as communicating through multiple layers of protocols:

- Application Protocol (e.g., HTTP, FTP)
- Transport Layer (TCP / UDP)
- Network Layer (IP)
- Data Link / Physical Layer

SECTION 2: THE TCP/IP PROTOCOL SUITE (THE BIG PICTURE)

Although the suite is called TCP/IP, it contains many more members. Each protocol is formally defined by one or more RFC (Request For Comments) documents.

2.1 All Protocols in the TCP/IP Family

Protocol	Full Name	Key Details
IPv4	Internet Protocol version 4	Uses 32-bit addresses. Provides packet delivery for TCP, UDP, SCTP, ICMP, IGMP.
IPv6	Internet Protocol version 6	Designed to replace IPv4. Uses 128-bit addresses. Provides packet delivery for TCP, UDP, SCTP, ICMPv6.
TCP	Transmission Control Protocol	Connection-oriented; reliable; full-duplex byte stream. Handles ACKs, timeouts, retransmissions, RTT estimation, sequencing, flow control, error control. RFC 793. Example: WWW, FTP, Email.
UDP	User Datagram Protocol	Connectionless; no guarantee of delivery; no retransmission; no reliability. Each datagram has a length. RFC 768. Example: VoIP, online games, media streaming.
SCTP	Stream Control Transmission Protocol	Connection-oriented; reliable full-duplex association. Supports multihoming (multiple IPs per side). Message-oriented (preserves record boundaries). Similar to both UDP and TCP.
ICMP	Internet Control Message Protocol	Handles error and control information between routers and hosts. Used by ping and traceroute. Normally processed by the TCP/IP stack, not user processes.
IGMP	Internet Group Management Protocol	Used with IP multicasting.
ARP	Address Resolution Protocol	Maps an IPv4 address to a hardware (MAC) address. Used on broadcast networks (Ethernet, Token Ring, FDDI). Not needed on point-to-point networks.
RARP	Reverse Address Resolution Protocol	Maps a hardware (MAC) address to an IPv4 address. Sometimes used when a diskless node boots.
BPF	BSD Packet Filter	Provides access to the data-link layer. Found on Berkeley-derived kernels. Used by tools like tcpdump.
DLPI	Datalink Provider Interface	Also provides access to the data-link layer. Normally provided with SVR4 (System V Release 4).

tcpdump NOTE

tcpdump communicates directly with the data link layer using either BPF (BSD Packet Filter) or DLPI (Datalink Provider Interface).

2.2 UDP — User Datagram Protocol (Detail)

- Simple transport-layer protocol (RFC 768)
- Application writes a message → encapsulated in UDP datagram → encapsulated in IP datagram → sent to destination

- No guarantee that UDP datagram reaches final destination (lack of reliability)
- No automatic retransmission when errors occur — BUT checksum detects errors; corrupted datagrams are dropped
- Each UDP datagram has a length — the length is passed to receiving application along with the data
- Connectionless — no long-term relationship between client and server
- Use cases: VoIP, online games, media streaming (applications that tolerate some loss)

2.3 TCP — Transmission Control Protocol (Detail)

- Provides connections between clients and servers (RFC 793)
- Provides RELIABILITY: when TCP sends data, it requires acknowledgment (ACK) in return
- Automatic retransmission in case of errors or lost segments
- RTT estimation: TCP contains algorithms to dynamically estimate round-trip time → knows how long to wait for ACK
- Sequencing: TCP assigns a sequence number to every byte it sends → enables reordering and duplicate detection at receiver
- Flow control: prevents fast sender from overwhelming slow receiver
- Error control: detects and recovers from errors
- Full-duplex: data can flow in both directions simultaneously
- Use cases: WWW (HTTP/HTTPS), FTP, Email (SMTP, IMAP)

2.4 SCTP — Stream Control Transmission Protocol (Detail)

- Provides services similar to both TCP and UDP
- Connection-oriented — provides associations between clients and servers
- Message-oriented (like UDP) — maintains record boundaries (unlike TCP which is byte-stream)
- Reliable full-duplex association
- Multihoming: an association can involve more than 2 IP addresses (multiple IPs per side of connection) — provides path redundancy

ASSOCIATION

In SCTP, a communication between 2 systems is called an 'association' (not a 'connection'). It may involve more than 2 IP addresses due to multihoming.

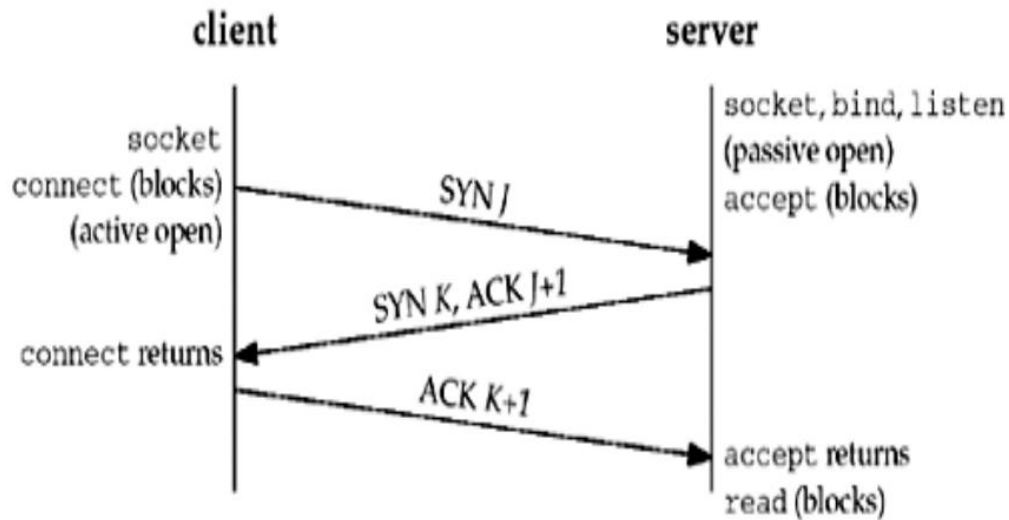
2.5 Difference Between UDP, TCP, and SCTP

Feature	UDP	TCP	SCTP
Connection type	Connectionless	Connection-oriented	Connection-oriented
Reliability	No — no guarantee	Yes — ACKs + retransmission	Yes — ACKs + retransmission
Data unit	Datagram (message)	Byte stream	Message (record boundaries preserved)
Ordering	No	Yes — sequence numbers	Yes
Flow control	No	Yes	Yes
Multihoming	No	No (single IP per end)	Yes — multiple IPs per end
Header size	8 bytes	20 bytes minimum	12 bytes minimum (common header)

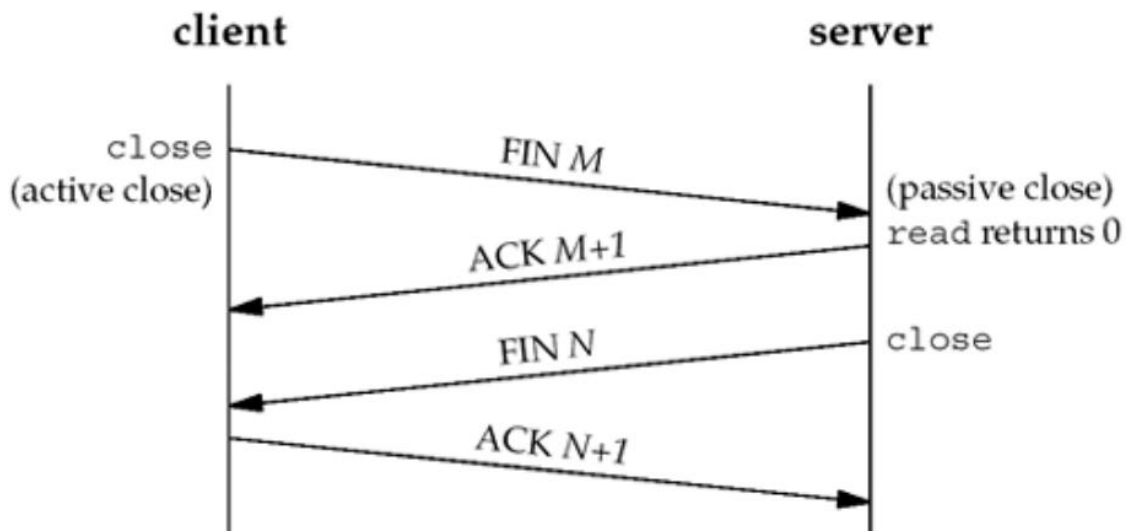
Use case	VoIP, DNS, games	HTTP, FTP, Email	Telecom signalling, reliable messaging
----------	------------------	------------------	--

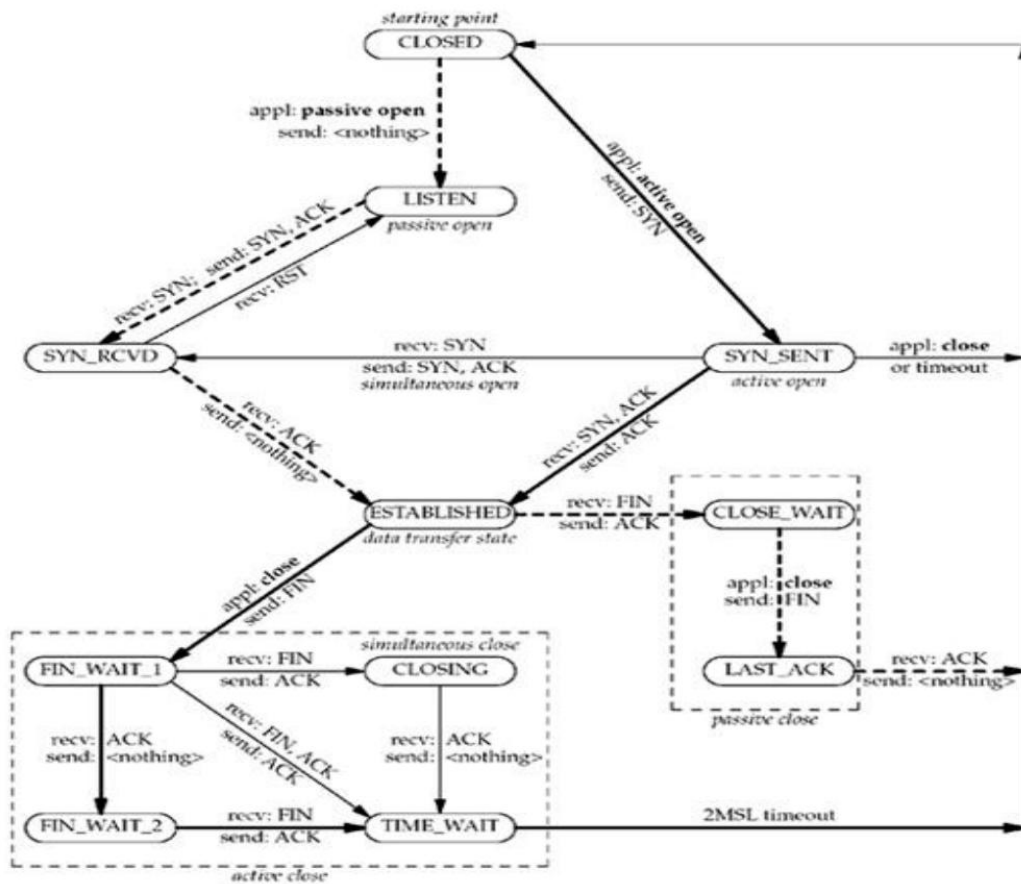
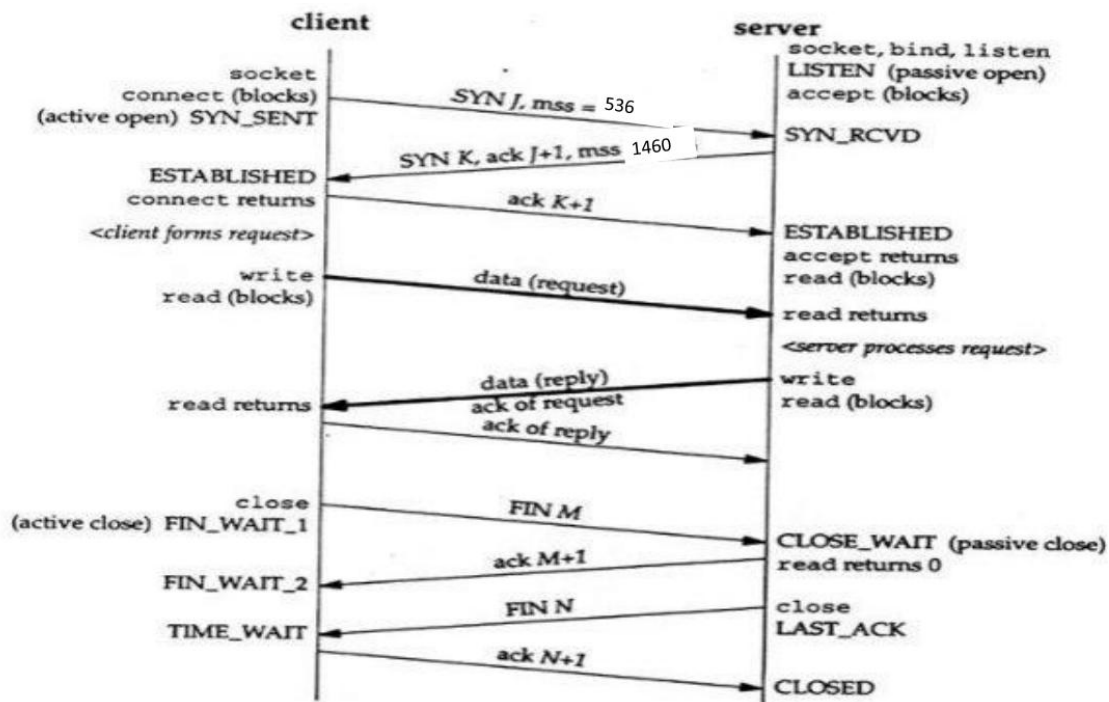
TCP Connection Establishment and Termination

Three-Way Handshake:



Four-Way Handshake:





SECTION 3: TCP CONNECTION — ESTABLISHMENT & TERMINATION

3.1 Three-Way Handshake (Connection Establishment)

Before any data can flow over a TCP connection, the connection must be established using a three-way handshake:

Step	Who	Action	Description
1 — SYN	Client	Sends SYN	Client performs ACTIVE OPEN by calling connect(). TCP sends a SYN segment containing client's Initial Sequence Number (ISN). No data is sent — only IP header + TCP header + possible options.
2 — SYN+ACK	Server	Sends SYN+ACK	Server performs PASSIVE OPEN (has called socket→bind→listen). Server ACKs client's SYN AND sends its own SYN with server's ISN. Both in a single segment.
3 — ACK	Client	Sends ACK	Client ACKs the server's SYN. Connection is now ESTABLISHED. Data transfer can begin.

SERVER SETUP

Server must call socket(), bind(), and listen() BEFORE the client calls connect(). This sets up the passive open.

3.2 Four-Way Handshake (Connection Termination)

TCP connection termination requires a four-way handshake because each direction of the connection must be closed independently:

Step	Who	Action	Description
1 — FIN	Active Closer	Sends FIN	One side (usually client) calls close() — performs ACTIVE CLOSE. Sends FIN segment meaning 'I am done sending data.'
2 — ACK	Passive Closer	Sends ACK	The other side ACKs the FIN. This ACK is also delivered to the application as an end-of-file (EOF) notification. Application will not receive more data.
3 — FIN	Passive Closer	Sends FIN	After the application calls close() on its end, its TCP sends a FIN.
4 — ACK	Active Closer	Sends ACK	Active closer ACKs the final FIN. Connection is fully closed (after TIME_WAIT).

HALF-CLOSE

After step 2, only one direction is closed (client→server). The server can still send data to the client (server→client still open) — this is called a half-close.

3.3 TCP State Transition Diagram

The TCP state machine tracks the state of a TCP connection. There are 11 states total:

#	State	Description
1	CLOSED	Initial / final state. No active connection.
2	LISTEN	Server has called listen(). Waiting for incoming SYN from client (passive open).
3	SYN_SENT	Client has sent SYN (called connect()). Waiting for SYN+ACK from server (active open initiated).
4	SYN_RECEIVED	Server received SYN, sent SYN+ACK. Waiting for ACK from client.
5	ESTABLISHED	Connection is open. Data transfer phase. Both sides can send/receive.
6	FIN_WAIT_1	Active closer has sent FIN. Waiting for ACK or FIN from other side.
7	FIN_WAIT_2	Active closer received ACK of its FIN. Waiting for FIN from other side.
8	CLOSE_WAIT	Passive closer received FIN, sent ACK. Waiting for application to close its end (call close()).
9	CLOSING	Simultaneous close — both sides sent FIN at same time. Rare case.
10	LAST_ACK	Passive closer has sent its FIN. Waiting for final ACK from active closer.
11	TIME_WAIT	Active closer received final FIN, sent final ACK. Waiting for 2×MSL before moving to CLOSED.

Connection Establishment State Flow

- CLOSED → SYN_SENT (client sends SYN — ACTIVE OPEN)
- CLOSED → LISTEN (server calls listen() — PASSIVE OPEN)
- LISTEN → SYN_RECEIVED (server receives SYN, sends SYN+ACK)
- SYN_SENT → ESTABLISHED (client receives SYN+ACK, sends ACK)
- SYN_RECEIVED → ESTABLISHED (server receives ACK — 3-way handshake complete)

Connection Termination State Flow — Active Close (Client closes first)

- ESTABLISHED → FIN_WAIT_1: client sends FIN
- FIN_WAIT_1 → FIN_WAIT_2: client receives ACK from server
- FIN_WAIT_2 → TIME_WAIT: client receives FIN from server, sends ACK
- TIME_WAIT → CLOSED: after 2×MSL timer expires

Connection Termination State Flow — Passive Close (Server side)

- ESTABLISHED → CLOSE_WAIT: server receives FIN, sends ACK
- CLOSE_WAIT → LAST_ACK: application calls close(); server sends FIN
- LAST_ACK → CLOSED: server receives final ACK

Special Case: Simultaneous Open

- Both sides send SYN at the same time — RARE case
- Both sides go to SYN_SENT, then both go to SYN_RECEIVED, then ESTABLISHED
- Leads through the CLOSING state

3.4 TIME_WAIT State

The TIME_WAIT state is entered by the side that performs the ACTIVE CLOSE (sends FIN first).

Duration

- $2 \times \text{MSL}$ (Maximum Segment Lifetime)
- MSL = 30 seconds to 2 minutes (implementation-dependent)
- Therefore TIME_WAIT lasts 1 to 4 minutes

Why TIME_WAIT Exists — Two Reasons

1. Reliable Connection Termination:
 - The final ACK sent by the active closer might get lost in the network
 - If the ACK is lost, the passive closer retransmits its FIN
 - TIME_WAIT ensures the active closer still remembers the connection and can resend the ACK
 - Without TIME_WAIT, a lost final ACK would cause the passive closer to get an RST error
2. Remove Old Duplicate Packets (Prevent 'Lost Duplicate' Problem):
 - Due to routing loops or network delays, old packets may still be wandering in the network
 - These old packets might arrive late and be mistakenly interpreted as part of a NEW connection on the same socket pair
 - TIME_WAIT waits for $2 \times \text{MSL}$ to ensure all old packets from the previous connection are gone before the socket address can be reused

EXAM TIP

TIME_WAIT is entered by the ACTIVE CLOSER only. Duration = $2 \times \text{MSL}$. Two purposes: (1) reliable termination via final ACK; (2) eliminate stale duplicate packets from old connections.

SECTION 4: PORTS, SOCKETS, AND SOCKET PAIRS

4.1 Port Numbers

- Port numbers are 16-bit unsigned integers: range 0 – 65535
- Used by TCP, UDP, and SCTP
- Purpose: identify which process/application should receive incoming data

IP address identifies the device (host). Port number identifies the application/process on that device.

Why Port Numbers Are Needed

Many applications run simultaneously on the same device (browser, WhatsApp, email client, etc.). Port numbers route incoming data to the correct application.

Server Ports vs Client Ports

	Server Ports	Client Ports
Type	Well-known / fixed port	Ephemeral port (temporary)
Assignment	Manually assigned — standard protocol	Automatically assigned by OS
Uniqueness	Fixed, known to everyone	Unique per connection
Examples	FTP=21, SSH=22, HTTP=80, HTTPS=443	Random high-numbered ports (e.g., 52184)
Reason	Clients need to know where to connect	Client does not care about its port number

Types of Port Numbers

Range	Type	Description / Examples
0 – 1023	Well-Known Ports	Reserved; assigned by IANA to standard services. FTP=21, SSH=22, Telnet=23, SMTP=25, DNS=53, HTTP=80, HTTPS=443, POP3=110, IMAP=143
1024 – 49151	Registered Ports	Used by applications; not strictly controlled. Example: MySQL=3306, PostgreSQL=5432, Windows File Sharing=6000–6063
49152 – 65535	Dynamic/Private Ports	Used by clients as ephemeral ports; temporary, assigned by OS for each connection; also called private ports

4.2 Sockets

Socket = IP Address + Port Number

DEFINITION

A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so the TCP layer can identify the destination application.

Example socket: 192.168.1.5 : 50000

4.3 Socket Pair (4-tuple)

A socket pair uniquely identifies a TCP connection. It consists of 4 values:

- { Local IP, Local Port, Remote IP, Remote Port }

Example: { 192.168.1.5 : 50000 , 142.250.190.78 : 80 }

- TCP: connection-based — uses full socket pair
- UDP: connectionless — uses ports for routing but no socket pair per se
- Multiple connections can exist at the same time; the socket pair ensures each is unique and no data mixup occurs

4.4 Multihoming

- If a device has multiple IP addresses (multihomed), one connection can have multiple socket pairs
- Seen in advanced protocols like SCTP which explicitly supports multihoming
- In SCTP, a single association can involve multiple IP addresses on each side

4.5 TCP Port Numbers and Concurrent Servers

What is a Concurrent Server?

A server that handles multiple clients simultaneously. It does this by creating a new child process for each client using `fork()`.

Initial Server State — Listening

- Server runs on a multihomed host (multiple IPs)
- Server binds to a well-known port (e.g., FTP port 21)
- Listening socket accepts connections on ANY local IP, port 21 (uses wildcard address)
- The wildcard address is specified using `INADDR_ANY` (value = 0)
- Client IP and port are not known yet at this stage

When a Client Connects

- Client details (IP, ephemeral port) become known when `connect()` is called
- Server calls `accept()` → creates a child process using `fork()`
- Child process handles the connected client using the new connected socket
- Parent process continues to listen for new connections on the original listening socket
- The socket pair uniquely identifies each client's connection

KEY RULE

The listening socket exists for the server's lifetime. A new connected socket is created by `accept()` for EACH client connection. When the client is done, the connected socket is closed.

Buffer Sizes and Limitations:

Certain limits affect the size of IP datagrams:

1. The maximum size of an IPv4 datagram is 65,535 bytes, including the IPv4 header. This is because of the 16-bit total length field where for an IPv6 datagram it is 65,535 bytes, including the 40-byte IPv6 header
2. Many networks have an MTU which can be dictated by the hardware. For example, the Ethernet MTU is 1,500 bytes.
3. The minimum link MTU for IPv4 is 68 bytes.
4. The smallest MTU in the path between two hosts is called the path MTU
5. When an IP datagram is to be sent out an interface, if the size of the datagram exceeds the link MTU, fragmentation is performed by both IPv4 and IPv6. The fragments are not normally reassembled until they reach the final destination.
6. If the "don't fragment" (DF) bit is set in the IPv4 header (Figure A.1), it specifies that this datagram must not be fragmented, either by the sending host or by any router.
7. IPv4 and IPv6 define a minimum reassembly buffer size, the minimum datagram size that we are guaranteed any implementation must support.
8. TCP has a maximum segment size (MSS) that announces to the peer TCP the maximum amount of TCP data that the peer can send per segment.
9. SCTP keeps a fragmentation point based on the smallest path MTU found to all the peer's addresses.

SECTION 5: BUFFER SIZES & LIMITATIONS

When an application sends data over the network, it passes through multiple layers (Application → TCP/UDP/SCTP → IP → Datalink). Each layer has size limits and buffers that affect how much data can be sent, how it is split, and whether fragmentation occurs.

5.1 Maximum Size of IP Packets

Protocol	Max Packet Size	Notes
IPv4	65,535 bytes	Limited by 16-bit Total Length field. Includes header + data.
IPv6	65,575 bytes	Standard. Includes 40-byte fixed header + data.
IPv6 Jumbogram	> 65,535 bytes	Uses Jumbo Payload option (32-bit length field). Used in high-speed networks only.

5.2 MTU — Maximum Transmission Unit

- MTU = maximum packet size that a network link can handle
- Ethernet MTU: 1500 bytes (most common)
- Minimum IPv4 MTU: 68 bytes
- Minimum IPv6 MTU: 1280 bytes

Path MTU

- The smallest MTU along the entire path from sender to receiver
- Since path can be different in each direction, MTU can vary per direction
- Important for determining if fragmentation is needed

5.3 Fragmentation

Fragmentation occurs when a packet's size exceeds the MTU of a network link it must traverse.

	IPv4	IPv6
Who fragments?	Both the sender (host) AND intermediate routers can fragment	ONLY the sender (source host) fragments. Routers do NOT fragment.
Where reassembled?	Only at the final destination (not at intermediate routers)	Only at the final destination
Result of too-large packet	Fragmentation occurs at routers or sender	Router sends ICMPv6 'Packet Too Big' back to sender

CRITICAL DIFF

IPv4: Routers CAN fragment packets. IPv6: Only the HOST/SENDER can fragment. Routers drop oversized IPv6 packets and send ICMPv6 'Packet Too Big' message.

SECTION 6: SOCKET ADDRESS STRUCTURES

Most socket functions require a pointer to a socket address structure as an argument. Each supported protocol suite defines its own socket address structure. Names begin with `sockaddr_` and end with a unique suffix for each protocol suite.

6.1 IPv4 Socket Address Structure — `sockaddr_in`

Defined in `<netinet/in.h>`. Commonly called the 'Internet socket address structure'.

```
struct in_addr {
    in_addr_t    s_addr;          /* 32-bit IPv4 address - network byte order */
};

struct sockaddr_in {
    uint8_t      sin_len;         /* length of structure (16 bytes) */
    sa_family_t  sin_family;      /* AF_INET */
    in_port_t     sin_port;       /* 16-bit TCP or UDP port - network byte order */
    struct in_addr sin_addr;      /* 32-bit IPv4 address - network byte order */
    char         sin_zero[8];     /* unused - must be set to zero */
};
```

Key Points About `sockaddr_in`

- `sin_len`: length field added in 4.3BSD-Reno for OSI protocol support. Not required by POSIX, but simplifies variable-length structure handling.
- POSIX requires only: `sin_family`, `sin_addr`, `sin_port` — but `sin_zero[8]` is almost always included to make structure 16 bytes
- `sin_port` and `sin_addr` MUST be in network byte order (big-endian)
- `sin_addr` can be accessed as: `serv.sin_addr` (`in_addr` structure) OR `serv.sin_addr.s_addr` (`in_addr_t = uint32_t`)
- `sin_zero` is UNUSED but always set to zero by convention (entire structure zeroed before filling in)
- Socket address structures are used only on the given host — the structure itself is NOT transmitted over the network (but fields like IP and port are)

6.2 Generic Socket Address Structure — `sockaddr`

Defined in `<sys/socket.h>`. Provides a generic type so socket functions can accept any address family.

```
struct sockaddr {
    uint8_t      sa_len;
    sa_family_t  sa_family;      /* address family: AF_XXX value */
    char         sa_data[14];    /* protocol-specific address */
};
```

Socket functions are defined to accept pointer to struct `sockaddr` (the generic type):

```
int bind(int, struct sockaddr *, socklen_t);
```

Therefore, when calling socket functions, you MUST CAST the protocol-specific structure pointer to the generic type:

```
struct sockaddr_in serv; /* IPv4 socket address structure */
/* fill in serv{} */
bind(sockfd, (struct sockaddr *) &serv, sizeof(serv));
/* cast required - otherwise compiler warning */
```

WHY CAST?

The socket API predates ANSI C's void generic pointer. The solution chosen in 1982 was to cast all address structures to struct sockaddr*. Without the cast, the compiler will warn about incompatible pointer types.*

6.3 IPv6 Socket Address Structure — sockaddr_in6{}

Defined in <netinet/in.h>. Required if SIN6_LEN is defined (indicates length field support).

```
struct in6_addr {
    uint8_t s6_addr[16]; /* 128-bit IPv6 address - network byte order */
};

#define SIN6_LEN /* required for compile-time tests */

struct sockaddr_in6 {
    uint8_t sin6_len; /* length of structure (28 bytes) */
    sa_family_t sin6_family; /* AF_INET6 */
    in_port_t sin6_port; /* port number - network byte order */
    uint32_t sin6_flowinfo; /* flow information (low 20 bits = flow label) */
    struct in6_addr sin6_addr; /* 128-bit IPv6 address - network byte order */
    uint32_t sin6_scope_id; /* scope zone identifier (e.g., interface index) */
};
```

Key Points About sockaddr_in6

- IPv6 family is AF_INET6 (vs AF_INET for IPv4)
- sin6_flowinfo: low-order 20 bits = flow label; high-order 12 bits = reserved
- sin6_scope_id: identifies scope zone — commonly interface index for link-local addresses
- Structure is 64-bit aligned to optimize 64-bit processor access to sin6_addr
- IPv6 extensions to sockets API defined in RFC 3493

6.4 New Generic Socket Address Structure — sockaddr_storage{}

A new generic structure defined as part of the IPv6 sockets API. Overcomes limitations of the old struct sockaddr. Defined in <netinet/in.h>.

```
struct sockaddr_storage {
    uint8_t ss_len; /* length (implementation-dependent) */
    sa_family_t ss_family; /* address family: AF xxx value */
    /* implementation-dependent elements to provide: */
    /* a) alignment for all socket address types supported */
    /* b) enough storage for any socket address type supported */
};
```

Two Differences from struct sockaddr

3. Provides the STRICTEST alignment requirement of all socket address types on the system
 4. Is LARGE ENOUGH to contain ANY socket address structure the system supports
- Fields are opaque — only ss_family and ss_len are accessible
 - Must be cast or copied to the appropriate specific structure based on ss_family

6.5 POSIX Datatypes Used in Socket Structures

Datatype	Description	Header
int8_t	Signed 8-bit integer	<sys/types.h>
uint8_t	Unsigned 8-bit integer	<sys/types.h>
int16_t	Signed 16-bit integer	<sys/types.h>
uint16_t	Unsigned 16-bit integer	<sys/types.h>
int32_t	Signed 32-bit integer	<sys/types.h>
uint32_t	Unsigned 32-bit integer	<sys/types.h>
sa_family_t	Address family of socket address structure	<sys/socket.h>
socklen_t	Length of socket address structure (normally uint32_t)	<sys/socket.h>
in_addr_t	IPv4 address (normally uint32_t)	<netinet/in.h>
in_port_t	TCP or UDP port number (normally uint16_t)	<netinet/in.h>

6.6 Comparison of Socket Address Structures

Structure	Protocol	Length	Address Size
sockaddr_in{}	IPv4 (AF_INET)	Fixed — 16 bytes	32-bit IPv4 address
sockaddr_in6{}	IPv6 (AF_INET6)	Fixed — 28 bytes	128-bit IPv6 address
sockaddr_un{}	Unix domain (AF_LOCAL)	Variable-length	Pathname (up to 104 bytes)
sockaddr_dl{}	Datalink (AF_LINK)	Variable-length	Interface name + link-layer address
sockaddr_storage{}	Any (AF_XXX)	Longest on system	Opaque — holds any type

SECTION 7: VALUE-RESULT ARGUMENTS

When a socket address structure is passed to a socket function, it is always passed BY REFERENCE (via pointer). The length of the structure is also passed as an argument. HOW the length is passed depends on the direction:

7.1 Process → Kernel (bind, connect, sendto)

Three functions pass a socket address structure FROM the process TO the kernel:

- bind(), connect(), sendto()
- Length is passed as an INTEGER VALUE (not a pointer)
- Kernel knows exactly how much data to copy from process to kernel

```
struct sockaddr_in serv;  
/* fill in serv{} */  
connect(sockfd, (SA *) &serv, sizeof(serv));  
/* ↑ integer size - VALUE */
```

7.2 Kernel → Process (accept, recvfrom, getsockname, getpeername)

Four functions pass a socket address structure FROM the kernel TO the process:

- accept(), recvfrom(), getsockname(), getpeername()
- Length is passed as a POINTER TO AN INTEGER — this is called a VALUE-RESULT argument
- On function call: the integer tells kernel the SIZE of the buffer (so kernel doesn't overflow it)
- On function return: the integer tells the process how much information kernel actually stored

```
struct sockaddr_un cli; /* Unix domain */  
socklen_t len;  
len = sizeof(cli); /* len is a VALUE on input */  
getpeername(unixfd, (SA *) &cli, &len);  
/* ↑ pointer to len - VALUE-RESULT */  
/* on return, len contains actual bytes stored by kernel */
```

VALUE-RESULT

A value-result argument is one that serves dual purpose: it's both an input value (tells kernel buffer size) AND an output result (tells process how much was stored). Kernel cannot write past the end of the structure.

Other value-result arguments in sockets API:

- Middle three arguments for select() function
- Length argument for getsockopt()
- msg_namelen and msg_controllen members of msghdr{} when used with recvmsg()
- ifc_len member of ifconf structure

SECTION 8: BYTE ORDERING FUNCTIONS

8.1 Little-Endian vs Big-Endian

A 16-bit (or 32-bit) integer is stored in memory in two ways:

Order	Definition	Where high-order byte stored	Example: 0x0102
Little-Endian	Low-order byte at LOW (starting) memory address	At address A+1 (higher address)	c[0]=0x02 (LSB first), c[1]=0x01
Big-Endian	High-order byte at LOW (starting) memory address	At address A (lower address)	c[0]=0x01 (MSB first), c[1]=0x02

- Host byte order: the byte order used by a specific system
- Examples: Intel x86/x86_64 = little-endian; SPARC, PowerPC, MIPS (in many modes) = big-endian
- NETWORK BYTE ORDER = BIG-ENDIAN (Internet protocols use big-endian for all multi-byte fields)

NETWORK BYTE ORDER

The Internet protocols (TCP/IP) always use BIG-ENDIAN byte ordering. Port numbers and IP addresses in socket address structures must always be stored in network byte order.

8.2 Byte Order Conversion Functions

Four functions to convert between host byte order and network byte order:

```
#include <netinet/in.h>

uint16_t htons(uint16_t host16bitvalue); /* host-to-network short (16-bit) */
uint32_t htonl(uint32_t host32bitvalue); /* host-to-network long (32-bit) */
/* Both return: value in NETWORK byte order */

uint16_t ntohs(uint16_t net16bitvalue); /* network-to-host short (16-bit) */
uint32_t ntohl(uint32_t net32bitvalue); /* network-to-host long (32-bit) */
/* Both return: value in HOST byte order */
```

Function Name Decoding

Letter	Meaning
h	host (the system's native byte order)
n	network (big-endian — Internet standard)
to	convert to (direction of conversion)
s	short → 16-bit value (e.g., TCP/UDP port number)
l	long → 32-bit value (e.g., IPv4 address)

**USAGE
RULE**

ALWAYS use these functions when putting values into socket address structures (htons for port, htonl for IP) and when reading them back (ntohs, ntohl). On big-endian systems they are null macros (no overhead).

Usage example:

```
struct sockaddr_in serv;  
bzero(&serv, sizeof(serv));  
serv.sin_family = AF_INET;  
serv.sin_port = htons(80); /* convert port 80 to network byte order */  
serv.sin_addr.s_addr = htonl(INADDR_ANY); /* wildcard IP address */
```

SECTION 9: BYTE MANIPULATION FUNCTIONS

These functions operate on multibyte fields WITHOUT interpreting the data as strings (they don't assume null-termination). Needed when manipulating fields like IP addresses that can contain zero bytes.

9.1 BSD Berkeley-Derived Functions (names begin with 'b')

Defined in <strings.h>. From 4.2BSD. Still provided by systems that support sockets.

```
#include <strings.h>

void bzero(void *dest, size_t nbytes);
/* Sets nbytes bytes to 0 in destination. Used to initialize socket address
structures. */

void bcopy(const void *src, void *dest, size_t nbytes);
/* Moves nbytes bytes from source to destination. Handles overlapping fields
correctly. */

int bcmp(const void *ptr1, const void *ptr2, size_t nbytes);
/* Compares two byte strings. Returns 0 if equal, nonzero if unequal. */
```

MOST USED

bzero is the most commonly used BSD function — it's used to zero out a socket address structure before filling it in. It has only 2 arguments (vs memset's 3).

9.2 ANSI C Standard Functions (names begin with 'mem')

Defined in <string.h>. Available on all systems with an ANSI C library.

```
#include <string.h>

void *memset(void *dest, int c, size_t len);
/* Sets len bytes to value c in destination. */

void *memcpy(void *dest, const void *src, size_t nbytes);
/* Copies nbytes from src to dest. UNDEFINED if src and dest overlap! */

int memcmp(const void *ptr1, const void *ptr2, size_t nbytes);
/* Compares two byte strings. Returns 0 if equal, >0 or <0 if unequal. */
/* Comparison treats bytes as unsigned chars. */
```

9.3 Comparison: BSD vs ANSI C Functions

BSD Function	ANSI C Equivalent	Key Difference
bzero(dest, n)	memset(dest, 0, n)	bzero: 2 args, always zeros. memset: 3 args, can set any value.
bcopy(src, dest, n)	memcpy(dest, src, n)	bcopy: src then dest. memcpy: dest then src (like assignment: dest = src). bcopy handles overlaps; memcpy does NOT.

bcmp(p1, p2, n)

memcmp(p1, p2, n)

bcmp: returns 0 or nonzero. memcmp: returns 0, <0, or >0.

**OVERLAP
WARNING**

If source and destination overlap, use memmove() (ANSI C) — NOT memcpy(). bcopy() handles overlaps correctly.

SECTION 10: ADDRESS CONVERSION FUNCTIONS

These functions convert Internet addresses between ASCII strings (human-readable presentation format) and network byte-ordered binary values (stored in socket address structures). Two groups exist:

- Group 1: `inet_aton`, `inet_addr`, `inet_ntoa` — IPv4 ONLY (older functions)
- Group 2: `inet_pton`, `inet_ntop` — works with BOTH IPv4 AND IPv6 (newer, preferred)

10.1 Old IPv4-Only Functions

```
#include <arpa/inet.h>

int inet_aton(const char *strptr, struct in_addr *addrptr);
/* Converts dotted-decimal string to 32-bit binary network byte order value. */
/* Returns: 1 if valid string, 0 on error. Stores result through addrptr. */

in_addr_t inet_addr(const char *strptr);
/* Same conversion. Returns the 32-bit value directly. */
/* DEPRECATED — problem: returns INADDR_NONE (all 1s) on error, but */
/* 255.255.255.255 (broadcast) is a valid address with the same value! */

char *inet_ntoa(struct in_addr inaddr);
/* Converts 32-bit binary IPv4 address to dotted-decimal string. */
/* Result stored in STATIC memory — NOT re-entrant! Takes a structure, not pointer. */
```

o

Problems with Old Functions

- `inet_addr` is DEPRECATED — cannot handle 255.255.255.255 (broadcast address) correctly
- `inet_ntoa` is NOT re-entrant (uses static buffer) — NOT thread-safe
- Both functions only work with IPv4
- New code should use `inet_aton` instead of `inet_addr`, and preferably use `inet_pton`/`inet_ntop`

10.2 New Functions — `inet_pton` and `inet_ntop` (IPv4 + IPv6)

'p' = presentation (human-readable string). 'n' = numeric (binary value in socket structure). These functions handle BOTH `AF_INET` (IPv4) and `AF_INET6` (IPv6).

```
#include <arpa/inet.h>

int inet_pton(int family, const char *strptr, void *addrptr);
/* Presentation → Numeric (string to binary) */
/* family: AF_INET or AF_INET6 */
/* Returns: 1 if OK, 0 if invalid format, -1 on error (errno = EAFNOSUPPORT) */

const char *inet_ntop(int family, const void *addrptr, char *strptr, size_t len);
/* Numeric → Presentation (binary to string) */
/* len: size of destination buffer (to prevent overflow) */
/* Returns: pointer to result string if OK, NULL on error (errno = ENOSPC if buffer too small) */
/* strptr CANNOT be null — caller must allocate memory */

/* Buffer size constants: */
#define INET_ADDRSTRLEN 16 /* for IPv4 dotted-decimal (e.g., 255.255.255.255\0) */
#define INET6_ADDRSTRLEN 46 /* for IPv6 hex string */
```

Migration from Old to New Functions

Old Code	New Code (inet_pton)
foo.sin_addr.s_addr = inet_addr(cp);	inet_pton(AF_INET, cp, &foo.sin_addr);
ptr = inet_ntoa(foo.sin_addr);	char str[INET_ADDRSTRLEN]; ptr = inet_ntop(AF_INET, &foo.sin_addr, str, sizeof(str));

10.3 Summary of All Address Conversion Functions

Function	Direction	Handles IPv4?	Handles IPv6?	Status
inet_aton()	string → binary	Yes	No	Older — use for IPv4 still OK
inet_addr()	string → binary	Yes	No	DEPRECATED — avoid
inet_ntoa()	binary → string	Yes	No	Older — not thread-safe
inet_pton()	string → binary	Yes (AF_INET)	Yes (AF_INET6)	PREFERRED — modern
inet_ntop()	binary → string	Yes (AF_INET)	Yes (AF_INET6)	PREFERRED — modern

10.4 sock_ntop and Related Helper Functions

inet_ntop requires the caller to know the address family and pass the correct pointer — making code protocol-dependent. sock_ntop solves this by accepting a generic socket address pointer and automatically determining the format.

```
#include "unp.h"

char *sock_ntop(const struct sockaddr *sockaddr, socklen_t addrlen);
/* Returns: non-null pointer if OK, NULL on error */
/* Internally calls inet_ntop with correct family based on sa_family */
/* Output format: IPv4 = "dotted-decimal:port", IPv6 = "[hex]:port" */
/* Uses static buffer — NOT re-entrant */
```

Other helper functions in this family (for protocol-independent code):

Function	Description
sock_bind_wild(sockfd, family)	Binds wildcard address and ephemeral port to socket
sock_cmp_addr(sa1, sa2, len)	Compares the address portion of two socket address structures; returns 0 if equal
sock_cmp_port(sa1, sa2, len)	Compares the port number of two socket address structures; returns 0 if equal
sock_get_port(sa, len)	Returns just the port number as a non-negative integer
sock_ntop_host(sa, len)	Converts only the HOST portion (not port) to presentation format
sock_set_addr(sa, len, ptr)	Sets only the address portion from value pointed to by ptr
sock_set_port(sa, len, port)	Sets only the port number in a socket address structure

<code>sock_set_wild(sa, len)</code>	Sets the address portion to the wildcard address
-------------------------------------	--

SECTION 11: readn, writen, AND readline FUNCTIONS

Stream sockets (TCP) have a behavior different from regular file I/O: a single `read()` or `write()` call may return FEWER bytes than requested — this is NOT an error. Reasons include kernel buffer limits.

- For read: this can always happen on a stream socket
- For write: normally seen when socket is non-blocking; but `writen()` is always used to be safe

Three wrapper functions are provided to handle this:

```
#include "unp.h"

ssize_t readn(int filedес, void *buff, size_t nbytes);
/* Reads exactly nbytes bytes from a descriptor. Loops if fewer bytes are returned.
*/

ssize_t writen(int filedес, const void *buff, size_t nbytes);
/* Writes exactly nbytes bytes to a descriptor. Loops if fewer bytes are written. */

ssize_t readline(int filedес, void *buff, size_t maxlen);
/* Reads from descriptor one byte at a time until newline or maxlen-1 bytes read. */

/* All return: number of bytes read/written, or -1 on error */
```

WHY THESE EXIST

Standard `read()` and `write()` on TCP sockets may do short reads/writes. `readn/writen` loop internally until the full requested byte count is transferred. Always use these instead of raw `read/write` on stream sockets.

SECTION 12: SOCKET FUNCTIONS (THE SOCKETS API)

The sockets API provides functions needed to write TCP/UDP client and server programs. A complete TCP server uses: socket → bind → listen → accept → read/write → close. A complete TCP client uses: socket → connect → read/write → close.

12.1 socket() — Create a Socket

To perform network I/O, the first thing a process must do is call socket().

```
#include <sys/socket.h>

int socket(int family, int type, int protocol);
/* Returns: non-negative socket descriptor (sockfd) on success, -1 on error */
```

Arguments

family — Protocol Family

family	Description
AF_INET	IPv4 protocols
AF_INET6	IPv6 protocols
AF_LOCAL (AF_UNIX)	Unix domain protocols (inter-process communication on same host)
AF_ROUTE	Routing sockets
AF_KEY	Key socket (for IPsec key management)

type — Socket Type

type	Description
SOCK_STREAM	Stream socket — byte stream (reliable, connection-oriented). Used by TCP.
SOCK_DGRAM	Datagram socket — datagrams (unreliable, connectionless). Used by UDP.
SOCK_SEQPACKET	Sequenced packet socket — reliable, connection-oriented, message-boundaries preserved. Used by SCTP.
SOCK_RAW	Raw socket — bypass transport layer; used by ping (ICMP), traceroute.

protocol — Protocol

protocol	Description
IPPROTO_TCP	TCP transport protocol
IPPROTO_UDP	UDP transport protocol
IPPROTO_SCTP	SCTP transport protocol
0	System selects default for given family+type combination

Valid Combinations

family	type	protocol	Result
AF_INET	SOCK_STREAM	IPPROTO_TCP or 0	TCP over IPv4
AF_INET	SOCK_DGRAM	IPPROTO_UDP or 0	UDP over IPv4
AF_INET	SOCK_SEQPACKET	IPPROTO_SCTP	SCTP over IPv4
AF_INET6	SOCK_STREAM	IPPROTO_TCP or 0	TCP over IPv6
AF_INET6	SOCK_DGRAM	IPPROTO_UDP or 0	UDP over IPv6
AF_INET6	SOCK_SEQPACKET	IPPROTO_SCTP	SCTP over IPv6

RETURN VALUE

socket() returns a non-negative integer socket descriptor (sockfd) on success — similar to a file descriptor. Returns -1 on failure.

12.2 connect() — Client Establishes Connection

The connect() function is used by a TCP client to establish a connection with a TCP server.

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
/* Returns: 0 if OK, -1 on error */
/* sockfd: socket descriptor from socket() */
/* servaddr: pointer to socket address structure with server IP + port */
/* addrlen: size of socket address structure */
```

Behavior

- For TCP sockets, connect() initiates the three-way handshake
- Function returns ONLY when connection is established OR an error occurs
- Client does NOT need to call bind() before connect() — kernel automatically assigns ephemeral port and source IP

Error Returns from connect()

Error	Cause	errno
No response to SYN	Server is down or unreachable. TCP retries (several times) then gives up.	ETIMEDOUT
RST received	Server's response to SYN is a RESET — no process listening on that port (server not running). Hard error — returned immediately.	ECONNREFUSED
ICMP Destination Unreachable	Client's SYN gets ICMP 'unreachable' from intermediate router. Soft error — TCP retries for some time.	EHOSTUNREACH or ENETUNREACH

STATE CHANGE

If connect() fails, the socket can no longer be used. You must close() it and create a new socket() before trying connect() again.

12.3 bind() — Assign Local Address to Socket

bind() assigns a local protocol address (IP address + port number) to a socket.

```
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
/* Returns: 0 if OK, -1 on error (errno = EADDRINUSE if port already in use) */
/* myaddr: pointer to socket address structure with local IP + port */
```

What You Can Specify with bind()

sin_port (port)	sin_addr (IP)	Result
0 (any ephemeral)	INADDR_ANY (wildcard)	Kernel chooses both port and IP address
0 (any ephemeral)	Specific IP	Kernel chooses port; socket bound to specific IP
Non-zero (specific)	INADDR_ANY (wildcard)	Specific port; kernel chooses IP when packet sent
Non-zero (specific)	Specific IP	Specific port AND specific IP address

Key Rules for bind()

- Servers BIND their well-known port when they start (so clients know where to connect)
- TCP clients normally do NOT call bind() — kernel assigns ephemeral port
- TCP servers should NOT let kernel choose ephemeral port (clients need a known port)
- Port 0: kernel assigns any available ephemeral port at bind() time
- INADDR_ANY (= 0): kernel picks IP when socket is connected (TCP) or when datagram is sent (UDP)

Wildcard Address and INADDR_ANY

```
/* IPv4 wildcard */
serv.sin_addr.s_addr = htonl(INADDR_ANY); /* kernel chooses IP — value 0 */

/* IPv6 wildcard */
serv6.sin6_addr = in6addr_any; /* kernel chooses IPv6 address */
```

EADDRINUSE

bind() returns EADDRINUSE ('address already in use') if trying to bind a port that is already in use, or if TIME_WAIT is in effect for that socket address. Use SO_REUSEADDR socket option to reuse addresses.

12.4 listen() — Mark Socket as Passive (Server Only)

listen() is called ONLY by a TCP server. It converts an unconnected socket into a passive socket.

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);
/* Returns: 0 if OK, -1 on error */
/* backlog: maximum number of connections to queue for this socket */
```

Two Actions Performed by listen()

5. Converts an unconnected socket into a PASSIVE socket — kernel accepts incoming connection requests for this socket
6. Specifies the BACKLOG — maximum number of connections the kernel should queue for this socket

listen() must be called after socket() and bind(), and BEFORE accept().

The Two Connection Queues

For every listening socket, the kernel maintains TWO queues:

Queue	Contents	TCP State	Description
Incomplete Connection Queue	SYNs received but 3-way handshake not yet complete	SYN_RCVD	Entry created when server receives client SYN and sends SYN+ACK. Entry removed when: (a) 3rd segment (client ACK) arrives → moved to completed queue, OR (b) entry times out.
Completed Connection Queue	Connections where 3-way handshake is complete	ESTABLISHED	accept() takes first entry from this queue. If empty, accept() blocks until an entry arrives.

The backlog Argument

- backlog = sum of BOTH queues (incomplete + completed)
- Some implementations multiply backlog by 1.5 for buffer
- Do NOT specify backlog = 0 — different implementations treat it differently; if you don't want connections, close() the listening socket
- Maximum backlog value is configurable (system-dependent)
- RTT determines how quickly entries are created and completed in the queues

SYN Flooding Attack

SYN flooding is a Denial of Service (DoS) attack that exploits the incomplete connection queue:

- Attacker sends SYNs at high rate to fill the incomplete queue
- Source IP addresses are SPOOFED (random) so server's SYN/ACK goes nowhere
- Spoofing also prevents server from knowing attacker's real IP
- Legitimate SYNs are not queued → legitimate clients cannot connect
- Defense: SYN cookies — kernel handles without allocating queue entries for each SYN

SYN FLOOD

SYN flood = attacker sends many SYNs with fake source IPs → fills incomplete queue → server cannot accept legitimate connections. IP spoofing is used to hide attacker's identity and exhaust server resources.

12.5 accept() — Accept a Connection (Server)

accept() is called by a TCP server to return the next completed connection from the completed connection queue.

```
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
/* Returns: non-negative connected descriptor if OK, -1 on error */
/* sockfd: the LISTENING socket (created by socket(), bind(), listen()) */
/* cliaddr: returns protocol address of connected peer (client IP + port) */
/* addrlen: VALUE-RESULT argument - size of cliaddr on input, actual on output */
```

What accept() Returns

- If completed queue is NOT empty: returns first completed connection immediately
- If completed queue IS empty: process blocks (sleeps) until a connection arrives
- Returns a NEW socket descriptor (connected socket) for the specific client
- Returns three values: new socket descriptor, client address (cliaddr), address length (addrlen)

Listening Socket vs Connected Socket

	Listening Socket	Connected Socket
Created by	socket() function	Automatically by kernel on accept()
Count	ONE per server (exists for server's lifetime)	ONE per connected client
Purpose	Accept new incoming connections	Communicate with specific client
First argument to accept()	Yes — first arg sockfd is the listening socket	No — it is the return value of accept()
Closed when?	Server shuts down	When client is done being served

KEY DISTINCTION

The listening socket (sockfd) is the first argument to accept(). The connected socket is the RETURN VALUE of accept(). These are DIFFERENT sockets. A server creates only ONE listening socket, but creates one connected socket PER CLIENT.

12.6 close() — Close a Socket

```
#include <unistd.h>

int close(int sockfd);
/* Returns: 0 if OK, -1 on error */
/* Default action: mark socket as closed, return to process immediately */
/* For TCP: initiates FIN sequence (active close) */
```

- close() marks the socket as closed and returns immediately to the calling process
- Sends FIN to initiate TCP four-way termination (active close)
- If other processes still have the socket open (via fork), the FIN is NOT sent until the last descriptor is closed

SECTION 13: fork() AND exec() FUNCTIONS

fork() and exec() are used to create concurrent servers — servers that handle multiple clients simultaneously.

13.1 fork() — Create a New Process

```
#include <unistd.h>

pid_t fork(void);
/* Returns: 0 in child process, non-zero PID of child in parent process, -1 on error
*/
```

Behavior of fork()

- fork() returns TWICE: once in the parent process, once in the child process
- In the PARENT: returns the PID (Process ID) of the newly created child (positive non-zero)
- In the CHILD: returns 0
- The return value is used to determine: 'Am I the parent or the child?'
- ALL file descriptors open in the parent BEFORE fork() are SHARED with the child AFTER fork() returns

How Concurrent Servers Use fork()

7. Server calls socket(), bind(), listen() — creates listening socket
8. Server calls accept() — blocks waiting for client
9. Client connects — accept() returns with connected socket
10. Server calls fork() — creates child process
11. Child process: closes the listening socket; handles the client (reads/writes connected socket); closes connected socket when done
12. Parent process: closes the connected socket; loops back to accept() for next client

SOCKET SHARING

After fork(), BOTH parent AND child have the connected socket open (descriptor is shared). Parent must close() its copy of the connected socket, and child must close() its copy of the listening socket.

Two Typical Uses of fork()

13. A process makes a copy of itself so each copy can handle a different operation simultaneously (concurrent server)
14. A process wants to execute another program: calls fork() to copy itself, then child calls exec() to replace itself with the new program

13.2 exec() — Execute a New Program

The ONLY way to execute a program file on disk in Unix is for an existing process to call one of the six exec functions.

- exec() replaces the current process image with the new program
- The new program starts at its main() function

- The PROCESS ID does NOT change
- File descriptors open before exec() normally remain open across exec()

The Six exec() Variants

```
#include <unistd.h>

int execl (const char *pathname, const char *arg0, ... (char *)0);
int execv (const char *pathname, char *const argv[]);
int execlp (const char *pathname, const char *arg0, ... (char *)0, char *const envp[]);
int execve (const char *pathname, char *const argv[], char *const envp[]);
int execlp (const char *filename, const char *arg0, ... (char *)0);
int execvp (const char *filename, char *const argv[]);
/* Only execve is a SYSTEM CALL. Other five are library functions calling execve. */
/* All return: -1 on error (no return on success) */
```

Differences Among the Six exec() Functions

Dimension	Options
Program file specification	l, v, lp, ve = PATHNAME (uses full path). p, vp = FILENAME (uses PATH environment variable; if contains '/', PATH is ignored)
Argument passing	execl, execlp (l = list): arguments passed one by one as separate args, terminated by null pointer. execv, execve, execvp (v = vector): arguments passed as array of pointers (argv[]), terminated by null pointer in array.
Environment	execl, execv, execlp, execvp: use current environment (external variable environ). execlp, execve: explicit environment pointer (envp[]) must be null-pointer terminated.

KEY FACT

Only execve() is a true system call. The other five exec variants are library functions that ultimately call execve(). The process ID does NOT change across exec().

SECTION 14: COMPLETE TCP CLIENT-SERVER FLOW

A summary of the complete sequence of socket function calls in a typical TCP client-server interaction:

14.1 Server Side (Sequential)

15. `socket()` → creates listening socket
16. `bind()` → assigns well-known port and IP address
17. `listen()` → marks socket as passive; sets backlog queue size
18. `accept()` → blocks until client connects; returns connected socket + client address
19. `read()` / `write()` → communicate with client using connected socket
20. `close()` → close connected socket; send FIN to client
 - Repeat from step 4 for next client (concurrent server: `fork()` after `accept()`)

14.2 Client Side

21. `socket()` → creates client socket (unconnected)
22. `connect()` → establishes TCP connection with server (3-way handshake). Kernel assigns ephemeral port automatically.
23. `write()` / `read()` → send request, receive response
24. `close()` → close socket; send FIN to server (active close)

14.3 Complete Timeline of TCP Communication

Phase	Server Action	Network Event	Client Action
Startup	<code>socket()</code> , <code>bind()</code> , <code>listen()</code>	—	<code>socket()</code>
Connection	<code>accept()</code> [blocks]	←— SYN —→	<code>connect()</code> [active open]
Connection	<code>accept()</code> [blocks]	— SYN+ACK —→	—
Connection	<code>accept()</code> returns	←— ACK —→	<code>connect()</code> returns
Data Transfer	<code>read()</code> [blocks]	←— Request —→	<code>write()</code>
Data Transfer	<code>write()</code>	— Response —→	<code>read()</code>
Termination	<code>read()</code> returns 0 (EOF)	←— FIN —→	<code>close()</code> [active close]
Termination	<code>write()</code> [if needed], then <code>close()</code>	— ACK —→	—
Termination	<code>close()</code> [passive close sends FIN]	— FIN —→	—
Termination	—	←— ACK —→	TIME_WAIT state, then CLOSED

SECTION 15: QUICK REVISION CHEATSHEET

Protocols Quick Reference

Protocol	Type	RFC	Key Feature
TCP	Transport	793	Connection-oriented, reliable, byte-stream, flow/error control, full-duplex
UDP	Transport	768	Connectionless, unreliable, datagram, no flow control — fast
SCTP	Transport	—	Connection-oriented, reliable, message-oriented, multihomed
IPv4	Network	791	32-bit address, fragmentation at host+routers
IPv6	Network	—	128-bit address, fragmentation at host ONLY
ICMP	Network	—	Error/control messages (ping, traceroute)
ARP	Link	—	IPv4 address → MAC address mapping
RARP	Link	—	MAC address → IPv4 address mapping

Socket Address Structures Quick Reference

Structure	Family	Size	Use Case
sockaddr_in{}	AF_INET	16 bytes (fixed)	IPv4 addresses + port numbers
sockaddr_in6{}	AF_INET6	28 bytes (fixed)	IPv6 addresses + port numbers
sockaddr{}	Any	16 bytes	Generic — used for casting in function calls
sockaddr_storage{}	Any	Largest on system	Protocol-independent — holds any address type
sockaddr_un{}	AF_LOCAL	Variable	Unix domain (IPC on same host)

Byte Ordering Quick Reference

Function	Direction	Size	Used For
htons()	Host → Network	16-bit (short)	Port numbers
htonl()	Host → Network	32-bit (long)	IPv4 addresses
ntohs()	Network → Host	16-bit (short)	Reading port numbers from structures
ntohl()	Network → Host	32-bit (long)	Reading IP addresses from structures

Address Conversion Quick Reference

Function	Direction	IPv4	IPv6	Note
inet_aton()	String → Binary	✓	✗	Returns 1/0; preferred over inet_addr

inet_addr()	String → Binary	✓	✗	DEPRECATED — can't handle broadcast addr
inet_ntoa()	Binary → String	✓	✗	NOT thread-safe (static buffer)
inet_pton()	String → Binary	✓	✓	PREFERRED. Returns 1/0/-1
inet_ntop()	Binary → String	✓	✓	PREFERRED. Needs length arg for safety

Socket Functions Quick Reference

Function	Who Calls	Purpose	Key Notes
socket()	Both	Create a socket	Returns sockfd. Args: family, type, protocol
bind()	Server (always); Client (optional)	Assign local address to socket	Server binds well-known port. Client usually skips (kernel assigns ephemeral port).
listen()	Server only	Mark socket as passive	Must call after bind(), before accept(). Sets backlog queue.
connect()	Client only	Establish connection	Initiates 3-way handshake. Client doesn't need to bind() first.
accept()	Server only	Accept client connection	Returns NEW connected socket + client address. Blocks if queue empty.
close()	Both	Close socket	Sends FIN for TCP. Returns immediately.
fork()	Server	Create child process	Returns twice. Shares open FDs. Used for concurrent servers.
exec()	Server child	Execute new program	6 variants. Only execve() is a syscall. PID unchanged.

TCP State Transition Quick Reference

State	When Entered	Next State(s)
CLOSED	Initial state; also after TIME_WAIT	LISTEN (server) or SYN_SENT (client)
LISTEN	After listen() is called	SYN_RECEIVED (on receiving SYN)
SYN_SENT	After connect() sends SYN	ESTABLISHED (on SYN+ACK) or CLOSED (on error)
SYN_RECEIVED	Server received SYN, sent SYN+ACK	ESTABLISHED (on ACK)
ESTABLISHED	Connection open — data transfer	FIN_WAIT_1 (active close) or CLOSE_WAIT (passive close)
FIN_WAIT_1	After sending FIN	FIN_WAIT_2 (ACK received) or CLOSING (simultaneous close)
FIN_WAIT_2	After receiving ACK of own FIN	TIME_WAIT (after receiving FIN from other side)
CLOSE_WAIT	After receiving FIN (passive close)	LAST_ACK (after application calls close())
CLOSING	Simultaneous close — rare	TIME_WAIT

LAST_ACK	After passive closer sends FIN	CLOSED (after receiving final ACK)
TIME_WAIT	Active closer after receiving final FIN	CLOSED (after 2×MSL timer)

Port Number Ranges Quick Reference

Range	Type	Used By
0 – 1023	Well-Known	Standard services: FTP(21), SSH(22), Telnet(23), SMTP(25), DNS(53), HTTP(80), HTTPS(443)
1024 – 49151	Registered	Application servers: MySQL(3306), PostgreSQL(5432)
49152 – 65535	Dynamic/Private	Client ephemeral ports — auto-assigned by OS

— End of Unit 1 Notes — Network Programming & Security —